4.Comandi e Iterazioni

Una dichiarazione specifica una o piu azioni che dovranno esser performate , come l'assegnazione di un valore a una variabile , passare un controllo a una funzione , saltare da una dichiarazione all'altra

Salti e loops sono delle dichiarazioni che controllano il flow del programma eccetto dei casi in cui le istruzioni di controllo risultano in salti , e queste vengono eseguite in modo sequenziale, cioe nell'ordine in cui appaiono nel programma

Un programma imperativo è un paradigma di programmazione che usa le dichiarazioni per cambiare lo stato del programma, invece un programma procedurale è sempre un paradigma di programmazione ma deriva dalla struttura del programma basata sul concetto di chiamata di funzione

C è sia un programma imperativo che procedurale

DEFINITION STATEMENTS

```
- usati per definire una variabile
- var definita = memoria creata
- devono avere nomi diversi se lo scopo è uguale
```

LOOPS

usiamo i loop per eseguire un gruppo di dichiarazioni, in c possiamo introdurre un loop alla volta tra queste :

- while
  - questa viene ripetuta finche una specifica condizione di controllo sia vera, se il programma è falso l'esecuzione del programma continua con l'istruzione che segue nel corpo del ciclo
- do while
  - do statement while (expression);
  - il loop interno del ciclo viene eseguito una volta prima del controllo dell'espressione controlla per la prima volta
  - questa assicura che venga eseguita almeno un iterazione del corpo
  - se è true continue se è false esce
- for
  - for ([expression1]; [expression2]; [expression3]) statement
  - 1. viene dichiarata una variabile che verra valutata prima dell'esecuzione del ciclo
  - 2 . controllo prima di ogni iterazione , se false termina
  - 3 .incremento contatore dell'espressione 1

SELECTION STATEMENTS

-Un istruzione di selezione ti puo indirizzare in una parte del programma in esecuzione a seconda del risultato scelto :

- if
  - if (expression) statement1 [else statement2]

    - else è un espressione opzionale
    - espressione valutata prima , deve avere un tipo scalare , se è vera e non uguale a 0 questa statement 1 viene eseguite , se non lo è esegue statement2
- switch
    - quest'istruzione ti permette durante l'esecuzione del programma di saltare da una parte all'altra di questa a seconda della scelta data
    - switch (expression) statement
    - l'espressione viene valutata 1 volta e compara con l'espressione presente nei vari case
    - se matcha con il case salta in quell'istruzione se non lo è esce, pero continuando l'esecuzione del programma

UNCONDITIONAL JUMPS STATEMENTS

queste interrompono l'esecuzione sequenziale delle istruzioni , modo che l'esecuzione d'essa avvenga in un altra parte di codice, e ne esistono di vari tipi:

- break
    - puo esser usata nei loop o nello switch e questa causa un salto nella prima parte presente dopo il loop o lo switch
    - usata anche per terminare l'esecuzione di un loop in qualsiasi esecuzione nel corpo d'essa
- continue
    - usata solamente dentro al corpo di un ciclo
    - nel do while salta nell'espressione successiva di controllo del ciclo
    - nel for salta nella 3 espressione del ciclo che contiene le operazione che vengono eseguite dopo ogni iterazione
- goto , and
    - il goto crea un incodizionabile salto in un altra espressione pero nella stessa funzione, la destinazione viene espressa con il nome delll'etichetta
    - goto label_name;
    - Un'etichetta è un nome seguito da due punti
    - nome_etichetta : istruzione
- return

TYPEDEF

il miglior modo per usare dei tipi con un nome complesso e quello di dichiararli semplicemente in modo anonimo per loro.

In una dichiarazione che inizia con la parola chiave typedef, ogni dichiaratore definisce un identificatore come sinonimo del tipo specificato.

L'identificatore viene quindi chiamato un "nome typedef" per quel tipo.

Tranne per la parola chiave typedef, la sintassi è esattamente la stessa di una dichiarazione di un oggetto o di una funzione del tipo specificato.